
3

PARADIGM OVERVIEW



The three paradigms included in this overview chapter are structured programming, object-orient programming, and functional programming.

STRUCTURED PROGRAMMING

The first paradigm to be adopted (but not the first to be invented) was structured programming, which was discovered by Edsger Wybe Dijkstra in 1968. Dijkstra showed that the use of unrestrained jumps (`goto` statements) is harmful to program structure. As we'll see in the chapters that follow, he replaced those jumps with the more familiar `if/then/else` and `do/while/until` constructs.

We can summarize the structured programming paradigm as follows:

Structured programming imposes discipline on direct transfer of control.

OBJECT-ORIENTED PROGRAMMING

The second paradigm to be adopted was actually discovered two years earlier, in 1966, by Ole Johan Dahl and Kristen Nygaard. These two programmers noticed that the function call stack frame in the ALGOL language could be moved to a heap, thereby allowing local variables declared by a function to exist long after the function returned. The function became a constructor for a class, the local variables became instance variables, and the nested functions became methods. This led inevitably to the discovery of polymorphism through the disciplined use of function pointers.

We can summarize the object-oriented programming paradigm as follows:

Object-oriented programming imposes discipline on indirect transfer of control.

FUNCTIONAL PROGRAMMING

The third paradigm, which has only recently begun to be adopted, was the first to be invented. Indeed, its invention predates computer programming itself. Functional programming is the direct result of the work of Alonzo Church, who in 1936 invented λ -calculus while pursuing the same mathematical problem that was motivating Alan Turing at the same time. His λ -calculus is the foundation of the LISP language, invented in 1958 by John McCarthy. A foundational notion of λ -calculus is immutability—that is, the notion that the values of symbols do not change. This effectively means that a functional language has no assignment statement. Most functional languages do, in fact, have some means to alter the value of a variable, but only under very strict discipline.

We can summarize the functional programming paradigm as follows:

Functional programming imposes discipline upon assignment.

FOOD FOR THOUGHT

Notice the pattern that I've quite deliberately set up in introducing these three programming paradigms: Each of the paradigms *removes* capabilities from the

programmer. None of them adds new capabilities. Each imposes some kind of extra discipline that is *negative* in its intent. The paradigms tell us what *not* to do, more than they tell us what *to* do.

Another way to look at this issue is to recognize that each paradigm takes something away from us. The three paradigms together remove `goto` statements, function pointers, and assignment. Is there anything left to take away?

Probably not. Thus these three paradigms are likely to be the only three we will see—at least the only three that are negative. Further evidence that there are no more such paradigms is that they were all discovered within the ten years between 1958 and 1968. In the many decades that have followed, no new paradigms have been added.

CONCLUSION

What does this history lesson on paradigms have to do with architecture? Everything. We use polymorphism as the mechanism to cross architectural boundaries; we use functional programming to impose discipline on the location of and access to data; and we use structured programming as the algorithmic foundation of our modules.

Notice how well those three align with the three big concerns of architecture: function, separation of components, and data management.